

UNIT-IV

Files: Physical storage devices and their characteristics, Attributes of a file viz fields, records, Fixed and variable length records, Primary and secondary keys, Classification of files, File operations, Comparison of various types of files, File organization: Serial, Sequential, Indexed-sequential, Random-access/Direct, Inverted, Multilist file organization. Hashing: Introduction, Hashing functions and Collision resolution methods

File

A collection of related records created as a unit known as file.

Constituents of a file

A file consists of a number of records. Each record is made up of a number of fields and each field consists of a number of characters.

Character

A character is a smallest element in a file and can be alphabetic, numeric or special.

Field

A group of characters form a field.

For example, Rollno, Registration number, name and marks are fields.

Sometimes a field may be divided into subgroups or subfields. For example, the marks field is divided into subfields such as sub1, sub2, sub3.

Record

A record is made up of a number of related fields.

e.g. A student record, an employee record etc.

deleted from the file. For example, deleting the record of a student who has left the school.

3. File Retrieval

File retrieval means the extraction of meaningful information from a file.

These are of two types:

Query

If retrieval is processed interactively, it is called query. It results in a relatively low volume data. It is generally done on-line.

For example, to find all the records where marks greater than 80 from the student file.

Report generation

If retrieval is processed in batch mode, it is called report generation. It results in many pages of output.

4. File Maintenance

Files are maintained to improve the performance. Two types of maintenance operations are:

Restructuring

Restructuring means changing the field structure. For example, field size can be changed, new field can be added.

Reorganizing

Reorganizing means changing from one file organization to another file organization.

Types of files

1. Master File
2. Transaction File
3. Dump File
4. Work File
5. Archive File
6. Library File

1. Master Files

- These files contain records that are relatively permanent in nature i.e. information is of value for long term and is used regularly in the organization's key systems.

- Master files are used for repeated processing. These should always be accurate and kept up to date. These are kept up to date through the use of transaction files. There can be number of master file for Payroll, a master file for Stocks, a master file for Accounts Payable etc.

Master files may be further classified as:

(a) Reference Master File

(b) Dynamic Master File

(a) Reference Master File

A reference master file is that in which records are static or unlikely to change frequently.

Examples:

- A product files containing descriptions and code
- An employee files containing empno, name, address, date of joining etc.
- A subject file containing a list of subjects offered inUniversity.

These files are used for referencing the data.

(b) Dynamic Master File

A dynamic master file contains records which are frequently changed/updated as a result of business transactions or other events.

Example:

A stock file containing itemcode, name andstocklevel, stocklevel changes every time there is an issue or receipt.

2. Transaction Files

These files contain records which are temporary in nature.

Transaction file has two purposes of:

- Accumulating data about events as they occur and
- Updating master files to reflect the result of current transactions.

For example, a store issue transaction is of active use only till the master file has been updated for the quantity issued.

Transaction files are erased or destroyed when no longer needed.

3. Dump Files

- These files contain copies of data held in the computer at a particular moment.
- A dump file is generally used for security purposes. It provides a backup, to permit recovery in case data is lost or damaged.
- It may be a complete copy of a master file to be retained to help recovery in the event of a possible future corruption of the master file.

4. Work File

A number of temporary files are created during the processing of a job. Such files are known as work files.

For example, to sort a large file, a number of work files may be used to provide the final sorted file.

5. Archive Files

- These files are used for long term storage of data.
- These are off-line files like transactions and master files that are no longer needed to be kept on-line.
- It may be used to draw conclusions from the recorded successes and failures of the organizations.

6. Library Files

- Library files are files which are not used very frequently.
- For example, files containing application programs, utility programs, system software packages and other useful data required may be kept as library files in the data processing center.

Serial File organization

- Serial file organization is the simplest organization of records in a file. In it, the records are placed one after the other serially on the storage medium.
- There is no specific order in the arrangement of records. Records are entered in the order in which they are presented.
- It can be implemented on both magnetic tapes and magnetic disks. On a magnetic tape, storage of the records is along the length of the tape while on the magnetic disk it is along the tracks, one after the other.

- This file organization is usually suitable for temporary files and then sorted into some sequence before being processed.
- For writing the records, almost all the storage capacity of the tape or disk is utilized. No space is left between the records. Records cannot be inserted in between; they can be added at the end of the file.
- In reading a serial file, the system always starts at the beginning of the file, and reads its way up to the record, one record at a time.
- For example, if a particular record happens to be the fifth one in a file, the system starts at the first one and reads ahead one record at a time until the fifth one is reached.

Advantage of serial file organization

Following are the advantages of serial file organization:

- Serial files are simple to understand and design.
- The main advantage of this file organization is that the space on the storage medium is utilized to the maximum.

Disadvantages of serial file organization

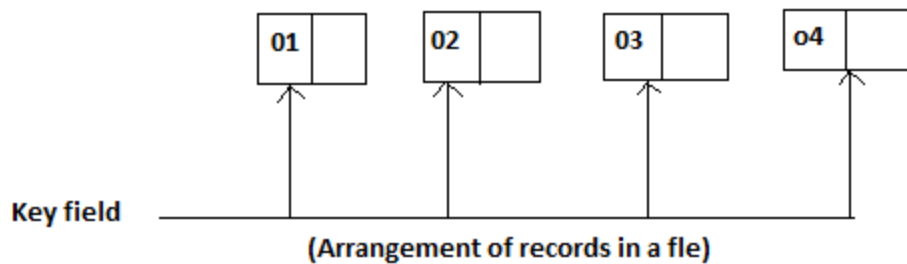
- To access the location of the desired record one has to search all the records serially till the location of the desired record is found.
- The average time required to locate a record in a file is nearly half of the total time to search the whole file.
- Records cannot be added to the middle of the file.

Sequential file organization

A sequential file organization is one in which the records are stored in a meaningful order according to a keyfield.

The records may be ordered in ascending or descending order by the keyfield which can be numeric or alphabetic.

A student file may be kept in the order of roll number as shown below:



- To access the name of roll number 4, 3 records in sequence have to be read and skipped.
- All the records can be identified by the keyfield rollnumber only.

Sequential access is possible on magnetic tapes. Sequential organization of records can also be implemented on a magnetic disk. Then these file can be processed in the same manner as files on magnetic tapes are processed.

Applications that use sequential files

Following are the applications that use sequential files:

- Sequential files are mostly used for commercial batch oriented data processing applications where details are added periodically. For example, Payroll applications.
- For applications where a file inquiry needs viewing or examination of every record in a file then the sequential access is best suited.
- For applications like searching a record in a file only a key-field is required. Locating a record is very easy as the file is in an ordered sequence.

Insertion operation in sequential files

- (i) Insertion of records at the end of sequential file i.e. Appending a sequential file

Appending a sequential file means adding more records to an existing file. Since the new record is added at the end of the existing records, the file is read record by record from the beginning till EOF is reached. Then the new record is written into the file.

The steps followed are:

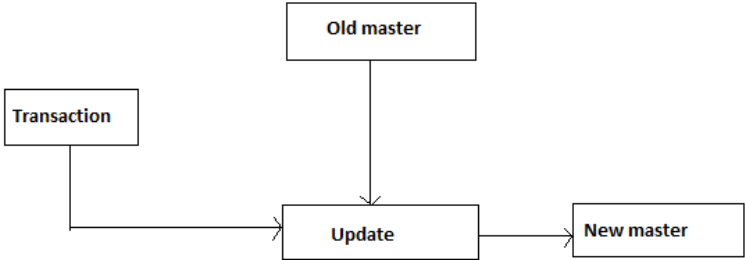
- (a) Open the file.

- (b) Read the file till EOF is reached.
 - (c) Read the record to be added.
 - (d) Write the record into the file.
 - (e) If want to add more record go to step (c).
 - (f) Close the file.
- (ii) Insertion of records at the desired sequence of key in a sequential file.
- Records may be inserted at the place dictated by the sequence of the key. Since direct insertions into the main data file leads to frequent rebuilding of the file. Thus the common method used is updating, which is described below:

Updating a file means modification of data values in a file.

- The original file is known as master file or old master file and the temporary file which contains the changes or transactions is known as transaction file.
- In order to update a sequential file, the transactions are sorted in the same key-field order as the original master file. A new file known as new master file is opened for writing the output.

For this process, the steps followed are:

- (a) A transaction record is read.
 - (b) An old master record is read,
- 
- ```
graph TD; Transaction[Transaction] --> Update[Update]; OldMaster[Old master] --> Update; Update --> NewMaster[New master]
```
- (c) The update program compares the key-fields of these two fields, if identical, the transaction record is written on the new master file otherwise the oldmaster record is written on the new master file.

**Deletion operation in sequential files**

The records to be deleted are written in the transaction file.

A new master file is created by copying all the records from the existing master file except the records kept in the transaction file.

The steps followed are:

- (a) Open new master file for output.
- (b) Open old master file for input.
- (c) While not EOF of old master file repeat step. (d) &(e)
- (d) Read master-record from old master file
- (e) If master-key # Del-key then

Write master-record on new master file.

- (f) Close the files

### **Steps to access a record of a file**

To access any record, the key-field of that record is entered. The system reads the file from the first record one by one till the desired record is reached or till the end of the file.

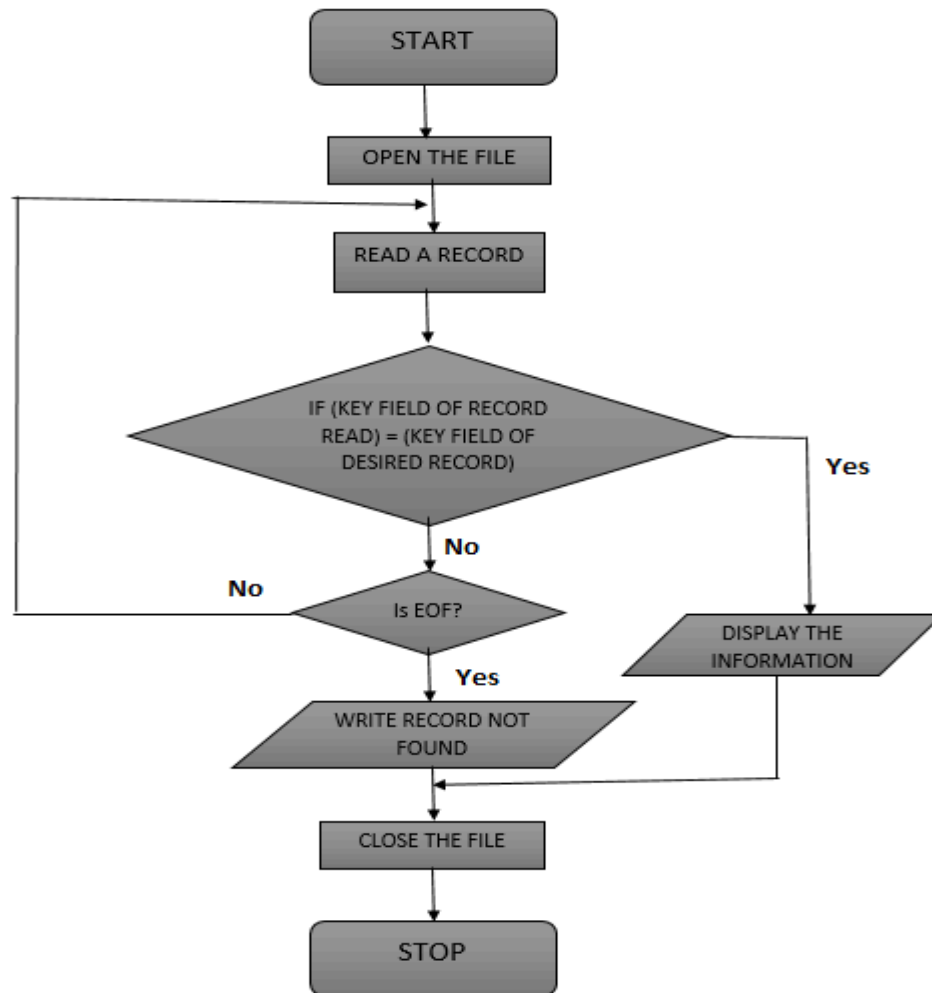
The end of file (EOF) indicates that all the records have been read.

Example: If a file consists of key-field from 1 to 500 and the record with the key-field 430 is to be accessed then it will be possible only after accessing the first 429 records.

To access a record of a file the steps followed are:

- (a) Open the file.
- (b) Read a record.
- (c) If the record is desired one, display the information and go to step (f).
- (d) If end of file reached display record not found, goto step (f).
- (e) Go to step (b).
- (f) Close the file.





### **Steps for updating a sequential file:-**

Updating a file means modification of data values in a file. It also includes operations like addition, deletion of records to a file.

The original file is known as master file or old master file and the temporary file which contains the changes or transactions is known as transaction file.

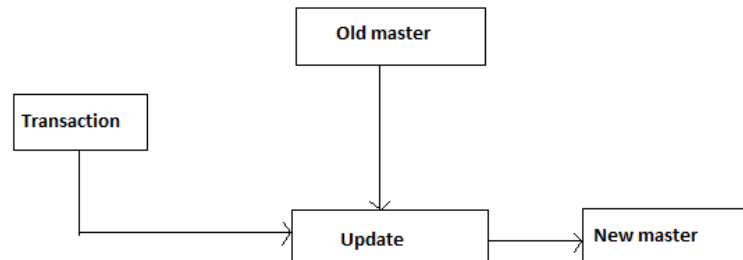
In order to update a sequential file, the transactions are sorted in the same key-field order as the original master file.

A new file known as new master file is opened for writing the output.

For this process, the steps followed are:

- (a) A transaction record is read.
- (b) An old master record is read.

(c) The update program compares the key-fields of these two fields, if identical, the transaction record is written on the new master file otherwise the old master record is written on the new master file.



## **Advantages and Disadvantages of sequential file processing**

### **Advantages of sequential file processing**

- The sequential file design is very simple and it is easy to create and implement.
- For searching a record in a file only a key-field is required.
- Locating a record is very easy as the file is in an ordered sequence.
- In case a file inquiry needs viewing or examination of every record in a file then the sequential access is best suited.
- Low cost devices like magnetic tapes are used. Thus it is an inexpensive way of data storage and security.

### **Disadvantages of sequential file processing**

- To access a particular record all its predecessors have to be scanned and skipped, causing a waste of time.
- Not suited for on-line inquiry or on-line processing where up to date information is required.

e.g. Banking applications, railway applications.

- Insertion, deletion is not easy because it may require moving a large proportion of the file.
- Overall processing is slow.

- It must be sorted before processing.’
- Entire file must be processed even if a single record is to be searched or updated.

## **Direct/Random access file organization**

- Random access file organization provides accessing the records directly. Each record has its own address, by the help of which it can be directly accessed for reading or writing.
- The records need not be in any sequence within the file and they need not be in adjacent locations on the storage medium.
- These files are created only on disk, since every record can be independently accessed. These types of files cannot be created on a magnetic tape medium.
- Best suited for on-line processing systems where current information is always required.
- For example, Airline ticketing system where a passenger can ask for booking or cancellation of his ticket.

### **Addressing of records**

There are three ways to address the records in a direct file. These are:

#### **1. Direct addressing**

In direct addressing the key-field is used as address field. It is implemented as:

$$\text{Key value} = \text{Address}$$

That is the actual address of record is the key value supplied for that record.

**Example:** Suppose a result file has records of students with primary key as roll number, then roll numbers can be used for addresses of the records.

The advantages of using this method are:

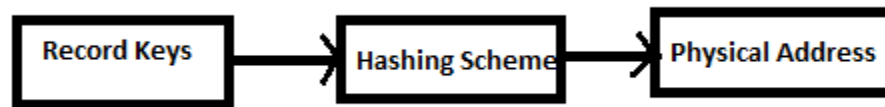
- Every record can be accessed in one seek and one read.
- The processing time to determine the record's location on secondary storage is minimum.

The only disadvantage of using this method is wastage of storage space because storage space has to be reserved for every value of the key-field. Thus this method results in low utilization of storage space most of the time.

#### **2. Indirect addressing**

The problem of low utilization is removed by using this method. In this method a transformation is made from the key of the record to the address of the storage

location at which that record is to reside in the file. The method that generates the address is called Hashing or Hashing scheme.



### **3. Index method addressing**

- This method uses a two dimensional array to find the addresses. The table has key-fields in one column and file addresses in the other column.
- To access a record, the table is searched for the key value and the address is used to access the record itself.

| Key | Address |
|-----|---------|
| 17  | 5       |
| 25  | 3       |
| 37  | 2       |
| 42  | 4       |
| 57  | 1       |
| 58  | 6       |

| Relative File |
|---------------|
| 57            |
| 37            |
| 25            |
| 42            |
| 17            |
| 58            |

### **Uses of direct access file organization**

Direct access file organization allows immediate direct access to individual records in a file. Thus, this file organization is useful whenever the application requires immediate access to individual records. It is widely useful these days as the computers display management **information on demand**.

### **Advantages of direct access file organization**

- More flexible as the records can be arranged both sequentially and non-sequentially.
- File processing & updating activities can be performed without the creation of new master file.
- Up to the minute information will be always available on the file.
- Direct access of records is possible.
- Addresses are obtained either by using hashing or hashing indexes.
- Insertion, deletion is easy.

### **Direct / random access file operations**

### **(a) Creating a direct file**

For creating a direct file a suitable hashing scheme used. In this scheme, the key value of input record transformed into physical address and then this record is stored at this physical address. This process continues until all the records are stored on the storage medium

### **(b) Accessing a direct file**

To access a record, user supplies the key value of the record. Using the hashing algorithm the physical address is calculated and then accessed. If there is no record at that address then the desired record is not present in the file.

### **(c) Updating a direct file**

Updating a file means modification of data values in a file. It also includes operations like addition, deletion of records to a file.

#### **i) Addition of a new record**

To add a new record its physical address is calculated using hashing scheme. If the calculated physical location is not vacant, collision-handling techniques are used to insert the record.

#### **ii) Deletion of a record**

Firstly the physical address of the record to be deleted is calculated using hashing scheme. If the record to be deleted does not exist in the file then display the error message otherwise delete it by making record-status code as vacant

#### **iii) Modification of a record**

Firstly the physical address of record to be modified is calculated using hashing scheme then modifications are made. The modified record is then rewritten in the file at the same location.

## **Indexed sequential file**

This method combines the advantages of sequential and direct file access. An indexed sequential file is basically a sequential file organized serially on a key field. In addition, an index is maintained which speeds up the access of records.

**For example,**

| <b>Key</b> | <b>Starting address of block</b> |
|------------|----------------------------------|
| 125        | 12                               |
| 860        | 19                               |
| 1420       | 24                               |
| 1600       | 42                               |

All records with index value up to 125 are in the first block on storage medium, which starts at the address 12 on the disk. If a record with key 1236 is required, it will be in the third block starting at address 24.

Many applications require indexed sequential file organization.

Example: In an Inventory control system at regular intervals, the transactions are processed sequentially in a file and if needed, make direct accesses inquiries into the same file. Since this file organization allows sequential as well as random access of records, these files can only be stored on direct access storage devices.

### **Organization of an Indexed sequential file**

Indexed sequential files are composed of three areas:

1. Prime area
2. Overflow area
3. Indexed area

#### **1. Prime Area**

When an indexed sequential file is created all the records are stored in the prime area. The records are stored in the order by the key field.

#### **2. Overflow Area**

When insertion of new record cannot be fitted into the prime area they are placed in the overflow area.

#### **3. Indexed area**

It contains indexes of records, which are used to locate a particular record for direct processing.

To understand the operations on index sequential file consider the following example:

Suppose a file consists of 20 records, organized sequentially on 4 tracks. Each track consists of 5 records as shown below:

|                |           |              |           |              |           |              |           |              |           |              |
|----------------|-----------|--------------|-----------|--------------|-----------|--------------|-----------|--------------|-----------|--------------|
| <b>Track 1</b> | <b>17</b> | <b>Rec17</b> | <b>21</b> | <b>Rec21</b> | <b>25</b> | <b>Rec25</b> | <b>27</b> | <b>Rec27</b> | <b>32</b> | <b>Rec32</b> |
| <b>Track 2</b> | <b>33</b> | <b>Rec33</b> | <b>34</b> | <b>Rec34</b> | <b>47</b> | <b>Rec47</b> | <b>52</b> | <b>Rec52</b> | <b>57</b> | <b>Rec57</b> |
| <b>Track 3</b> | <b>69</b> | <b>Rec69</b> | <b>70</b> | <b>Rec70</b> | <b>71</b> | <b>Rec71</b> | <b>72</b> | <b>Rec72</b> | <b>73</b> | <b>Rec73</b> |
| <b>Track 4</b> | <b>74</b> | <b>Rec74</b> | <b>75</b> | <b>Rec75</b> | <b>76</b> | <b>Rec76</b> | <b>79</b> | <b>Rec79</b> | <b>87</b> | <b>Rec87</b> |

The figure given below shows the file index provides the highest key-field of the records written on a track.

| Track No. | Highest key on track |
|-----------|----------------------|
| 1         | 32                   |
| 2         | 57                   |
| 3         | 73                   |
| 4         | 87                   |

When an index sequential file is created all the records are written into the prime area in sequence by the key field. The indexes of the tracks are generated by the operating system at this time.

To access any record say 71, one way is to read the file sequentially starting from first record 17 until the record 71 is reached. This is a very slow process.

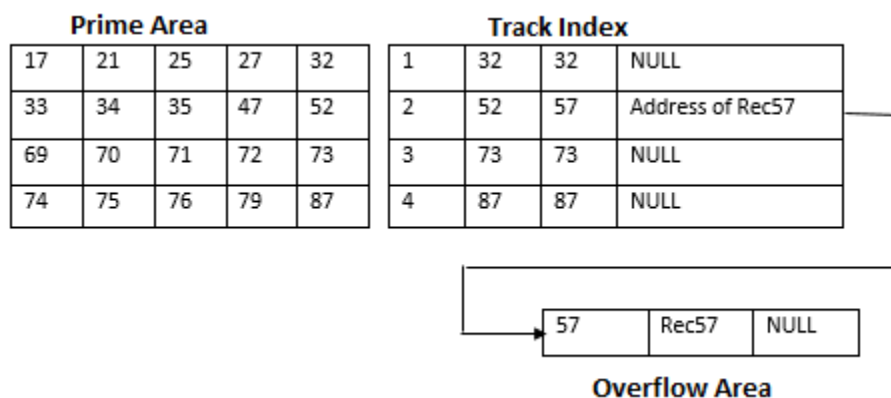
A faster way of finding record 71 is the use of track index. The highest key of track 3 is 73. The operating system starts reading track 3 sequentially till the record 71 is accessed. Thus this reduces the magnitude of the task instead of searching a large file of records.

### **Overflow handling techniques in indexed sequential file organization**

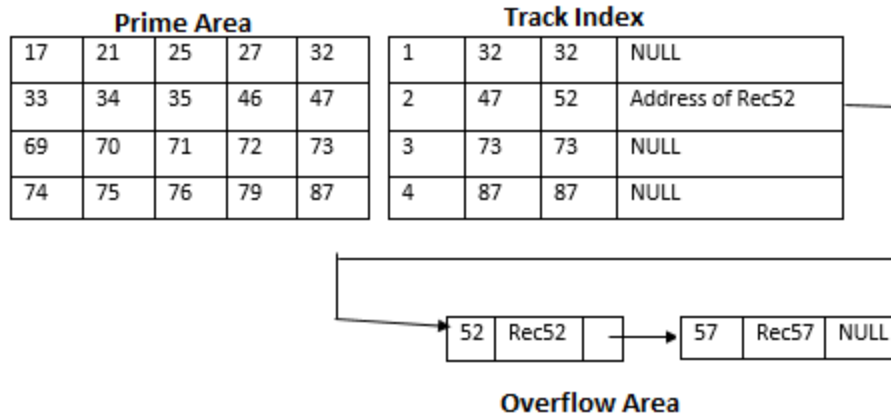
Consider insertion of record 35 in the file shown above. The record 35 has to be placed next to the record 34 and before the record 47 on track2.

If we write record 35 on top of record 47, record 47 on 52, record 52 on 57, record 57 on record 69 and so on till the end of file. This results in almost rewriting the rest of file.

The operating system writes record 35 on 47, 47 on 52, 52 on 57 and the record 57 is sent to the overflow area as shown below:



Insertion of record 46 to the file is shown below:



## **Multi-list file Organization**

A multi-list file organization is that in which records having equivalent values for a given secondary index items are linked together to form a list.

In other words, to reduce the number of records searched during a secondary key retrieval is to add a link field for each secondary key field for which retrieval is likely. Such a link field chains together all the records that have the same value in the associated data field. Each secondary key field for which a link field has been added has an array of list heads associated with it; each array contains as many entries as there are values for that field in the data records in the primary file. A list head points to the initial record in the primary file with a certain value. The link field in that record in turn points to the next record, which contains the same secondary key value.

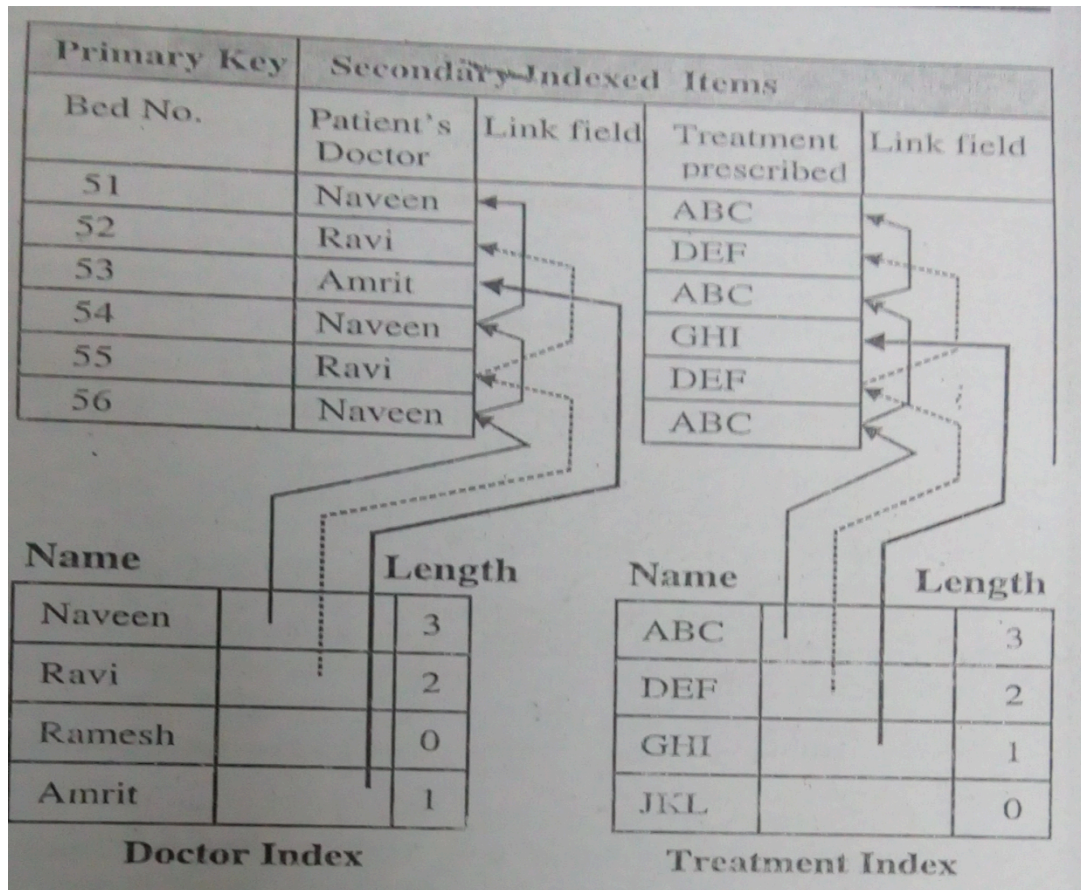
Suppose, a particular item may assume a number of values, say  $n$ , then  $n$  lists are created one for each item value.

It maintains an index for each secondary key. In the figure, there are two secondary keys Patient's doctor and Treatment prescribed.

Each entry in the index contains three fields:

- (i) The name of item value.
- (ii) A link to the first record in the list of records containing that item value and
- (iii) The length of the list.





The length field is extremely important as in the example it tells how many patients are under the treatment of Dr. Naveen and hence the popularity of doctor.

### Advantages of multi-list file organization

- The main advantage is simplicity of programming and the flexibility in updation.
- Quick access to every individual record is possible.

### Disadvantages of multi-list file organization

- The main disadvantage of the multi-list file organization is that in order to respond to a query, all corresponding records with all their lists must be brought into main memory for examination.
- Some amount of memory gets consumed in maintaining the link or address field.

### Inverted list file organization

Inverted file organization is that which provides an inverse relationship.

Example: Given a particular name, corresponding course can be located.

| Student's Name | Roll No. |
|----------------|----------|
| Ravi           | 101      |
| Mohan          | 102      |
| Hari           | 103      |
| Harvinder      | 104      |
| Ravinder       | 105      |
| Tajveer        | 106      |
| Harjinder      | 107      |
| Lalit          | 108      |
| Anil           | 109      |
| Faiz           | 110      |

| Course      | Roll No. |
|-------------|----------|
| Medical     | 101      |
|             | 110      |
|             | 103      |
|             | 107      |
| Non-Medical | 105      |
|             | 109      |
|             | 102      |
| Arts        | 108      |
|             | 104      |
|             | 106      |

The main advantage of this organization is its ability to handle queries.

- For example, the query "Is Ravinder an Arts student?" can be answered by first locating Ravinder's Roll No in the student's name list. Then Roll Number of all students in Arts course.
- The main disadvantage of this file organization is that the item values being inverted have to be included in both the lists.

## **Fixed and variable length records**

### **Fixed length record**

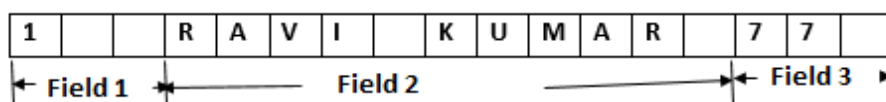
Records, which have corresponding fields in the records with the same field identical fields with lengths, are called fixed length records

For each of the corresponding fields of the records, a fixed storage space is allocated. A field may or may not fill the allocated storage space. Thus in fixed length records, each record is allocated the same fixed storage space.

Consider the student record divided into three fields as given below:

| Data Field | Field Length |
|------------|--------------|
| Roll_no    | 3            |
| Name       | 12           |
| Marks      | 3            |

The storage of this record as a fixed length record is shown below:



A lot of storage space is wasted while storing this 18 bytes long record as a fixed length record. Though storage of records as fixed length record results in wastage of storage space but these are more convenient to store and process.

### **Variable length record**

- Records which do not have the same lengths are called variable length records.
- To overcome the problem of wastage of the storage space variable length records are used. In variable length records, the various data fields are allowed only to storage space which they need.
- Here, the position of the end of each field and record is not known. So some special symbols are used to indicate these positions. The method of storage of a variable length record is shown in the following figure
- The character (,) indicates the end of a field in the record and the character (\*) signals the end of the record. Such symbols are called delimiters.

### **Primary and secondary keys:-**

#### **Primary keys**

A field in a record which identifies a record uniquely is called a primary key.

Primary keys give an ID to a row. They make the row unique throughout the entity. This means that rows can easily located by this identifiers. Primary keys can only be used for columns or attributes that do not allow null values. Allowing null values would mean that a row would not be uniquely identified.

Example:- consider the employee file:

Employee

| Emp_no | Emp_name | Designation | Salary |
|--------|----------|-------------|--------|
| 101    | Ravi     | Clerk       | 6050   |
| 102    | Hari     | Clerk       | 6250   |
| 103    | Mohan    | Assistant   | 9000   |
| 104    | Ravi     | Assistant   | 9000   |
| 105    | Ravinder | Assistant   | 9200   |

#### **Secondary key**

A field in a record which identifies the records but not uniquely is called a secondary key.

So, secondary keys are fields that store data that may or may not be unique. A secondary key may be either a single key or compound key. Secondary keys are used to retrieve data that may be common to a group of records.

### Example:-

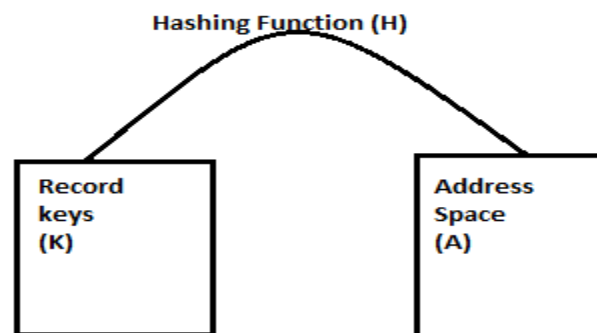
In the above employee file, the fields emp\_name, designation and salary are secondary keys.

## Hashing

Hashing is the transformation of a key into the corresponding location in the hash table. It deals with the idea of proving the direct address of the record where the record is likely to store.

### Hashing Function

Hashing function (H) is a mathematical formula that transforms the key space (K) of records to be the address space (A) of the records in a direct file.



A large number of hashing functions are available with the computer professionals. The hashing function selected must be such that corresponding to each record key there should be unique address where the record is to be placed.

So far no such hashing scheme has been developed that generates unique addresses. If different record keys generate the same address then a collision occurs and there are different collision handling techniques.

### Desirable characteristics of a hashing function are:

- The hash function should be easy and quick to compute.
- The hash function should distribute the keys to the number of locations of hash table with less number of collisions.

### Types of Hashing functions

Some of the common hashing functions are:

#### 1. Division/Remainder method

In this method, the keys are divided by a suitable number and the remainder is used for addressing the records.

**Example:** The address of record number 75481 would be calculated as follows:  
 $75481/1000 = 75$ , remainder 481.                      Address = 481

### **Merits**

- It is simple and easy to implement.
- It is very efficient for small files.

### **Demerits**

- It offers no guarantee against collision.
- As the size of the data set increases, so does the likelihood of collision.

## **2. Mid square method**

In this method

- Square the key value of the record.
- Extract a suitable number of digits from the middle of the squared value to get the address of the record on the disk.

**Example:** Let there be a file with 80 records then 2 digits address is required. Let the key value of the record be 583, then square of  $583 = 339889$ . Now drop the first two and last two digits of this number. We get 98. So physical address of the location of the record will be 98 which is the middle digits of the square of the key value.

### **Merits**

- This method avoids the use of division. Since integer division is usually slower, by avoiding this, one can potentially improve the running time of the hashing algorithm.
- It is simple and easy to implement.
- It is very efficient for small files as well as large files.

### **Demerits**

- This technique has one significant disadvantage: it offers no guarantee against collision but the performance is better than the division-remainder method.

## **3. Truncation method**

- Truncation method is also a division method which uses powers of 10 for division. Suppose a digit's key field is to be converted into 4 digit addresses, then the rightmost 4 digits of the keys can be used as addresses.
- Thus if the key-field is 234658329, then the address of the key is 8329.
- Truncation gives desirable results if the keys are continuous.

## **4. Extraction method**

- In this method any suitable group of digits can be extracted from the key-field.
- Let the given key value be 987654321, the three-digit address can be extracted from the middle to give address as 654.

- The extraction need not be a continuous string of digits. The address can be formed by extracting the second, fourth and seventh digit, giving the address as 863. Extraction is better than truncation method.

## **5. Folding method**

In this method the key is split into pieces and suitable arithmetic operations are done on the pieces.

Examples:

- **Folding**

Key: 285652

Extraction & addition:  $285 + 652$

Address: 937

- **Folding with remainder/division**

Key: 198765432

Extraction & addition:  $198 + 765 + 432$

Sum: 1395 (Divide it by 1000)

Address: 395 (Remainder)

- **Boundary folding with remainder/division**

In this method reverse the digits from the left half of the key before addition, which is known as boundary folding.

Key: 289649

Leftmost 3 digits: 289

Reverse number: 982

Addition:  $982 + 649 = 1631$

Address: 631.

## **Collision**

The situation in which the hash function returns the same hash key for more than one record is called collision.

Actually, the main problem in applying hashing to practical situation is the phenomenon of collision. It is a condition which arises when two or more keys are assigned the same home address

$$h(x_1) = y$$

$$h(x_2) = y$$

Collision can be avoided by having a powerful hashing algorithm which can spread the records uniformly across the available memory or the memory itself can be improved so that there is more space for accommodating records. Whatever may be the case, we have to devise methods to handle collisions.

### **Collision-handling techniques**

If collisions occur then it should be handled by applying some technique. Such a technique is called collision handling technique.

**There are two types of such techniques:**

1. Open addressing
2. Chaining

#### **1. Open addressing**

In open addressing if the address location to which the key is hashed is free, the element is stored at that location. In case it is already filled, then other address locations are examined systematically to find a free address location. If no such address location is found overflow condition occurs. The process of examining the address locations is called probing.

**Open addressing collision handling methods are:**

- (i) Linear probing
- (ii) Quadratic probing
- (iii) Double hashing

#### **(i) Linear probing**

When two records demand for the same location in the hash table, then this collision can be solved by placing second record linearly down wherever the empty location is found.

The hash function used for linear probing is:

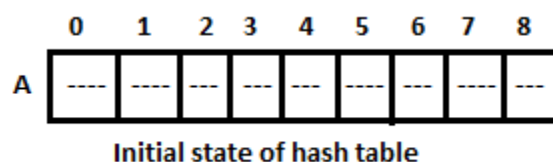
$$h(k, i) = [k \bmod m + i] \bmod m$$

$$\text{for } i = 0, 1, 2, \dots, m-1.$$

Where

- $m$  is the size of the hash table.
- $i$  is the probe number.

**Example:** Insert the keys 65, 22, 31, 49, 46, 98, 87, 12, 97 into a hash table of size  $m = 9$ .



**Step 1:-** Consider first key  $k = 65$

$$\begin{aligned}h(65,0) &= (65 \bmod 9 + 0) \bmod 9 \\&= (2+0) \bmod 9 \\&= 2\end{aligned}$$

Since address space  $A[2]$  is vacant, insert key 65 at this place.

|   | 0   | 1   | 2  | 3   | 4   | 5   | 6   | 7   | 8   |
|---|-----|-----|----|-----|-----|-----|-----|-----|-----|
| A | --- | --- | 65 | --- | --- | --- | --- | --- | --- |

Hash table after inserting key 65

**Step 2:-** Consider second key  $k = 22$

$$\begin{aligned}h(22,0) &= (22 \bmod 9 + 0) \bmod 9 \\&= (4 + 0) \bmod 9 \\&= 4\end{aligned}$$

Since address space  $A[4]$  is vacant, insert key 22 at this place.

|   | 0   | 1   | 2  | 3   | 4  | 5   | 6   | 7   | 8   |
|---|-----|-----|----|-----|----|-----|-----|-----|-----|
| A | --- | --- | 65 | --- | 22 | --- | --- | --- | --- |

Hash table after inserting key 22

**Step 3:-** Consider third key  $k = 31$

$$\begin{aligned}h(31,0) &= (31 \bmod 9 + 0) \bmod 9 \\&= (4+0) \bmod 9 \\&= 4\end{aligned}$$

Since address space  $A[4]$  is already occupied, the next address space is calculated as:

$$\begin{aligned}h(31, 1) &= (31 \bmod 9 + 1) \bmod 9 \\&= (4 + 1) \bmod 9 \\&= 5\end{aligned}$$

Since address space  $A[5]$  is vacant, insert key 31 at this place.

|   | 0   | 1   | 2  | 3   | 4  | 5  | 6   | 7   | 8   |
|---|-----|-----|----|-----|----|----|-----|-----|-----|
| A | --- | --- | 65 | --- | 22 | 31 | --- | --- | --- |



Hash table after inserting key 31

**Step 4:-** Consider fourth key  $k = 49$

$$\begin{aligned}h(49,0) &= (49 \bmod 9 + 0) \bmod 9 \\&= (4 + 0) \bmod 9 \\&= 4\end{aligned}$$

Since address space  $A[4]$  is already occupied, the next address space is calculated as:

$$\begin{aligned}h(49,1) &= (49 \bmod 9 + 1) \bmod 9 \\&= (4 + 1) \bmod 9 \\&= 5\end{aligned}$$

Since address space  $A[5]$  is already occupied, the next address space is calculated as:

$$\begin{aligned}h(49,2) &= (49 \bmod 9 + 2) \bmod 9 \\&= (4 + 2) \bmod 9 \\&= 6\end{aligned}$$

Since address space  $A[6]$  is vacant, insert key 49 at this place.

|   | 0   | 1   | 2  | 3   | 4  | 5  | 6  | 7   | 8   |
|---|-----|-----|----|-----|----|----|----|-----|-----|
| A | --- | --- | 65 | --- | 22 | 31 | 49 | --- | --- |

Hash table after inserting key 49

**Step 5.-** Consider fifth key  $k = 46$

$$\begin{aligned}h(46,0) &= (46 \bmod 9 + 0) \bmod 9 \\&= (1 + 0) \bmod 9 \\&= 1\end{aligned}$$

Since address space  $A[1]$  is vacant, insert key 46 at this place.

|   | 0   | 1  | 2  | 3   | 4  | 5  | 6  | 7   | 8   |
|---|-----|----|----|-----|----|----|----|-----|-----|
| A | --- | 46 | 65 | --- | 22 | 31 | 49 | --- | --- |

Hash table after inserting key 46

**Step 6:-** Consider sixth key  $k = 98$

$$\begin{aligned}h(98,0) &= (98 \bmod 9 + 0) \bmod 9 \\&= (8 + 0) \bmod 9\end{aligned}$$

$$= 8$$

Since address space A[8] is vacant, insert key 98 at this place.

|   | 0   | 1  | 2  | 3   | 4  | 5  | 6  | 7   | 8  |
|---|-----|----|----|-----|----|----|----|-----|----|
| A | --- | 46 | 65 | --- | 22 | 31 | 49 | --- | 98 |

Hash table after inserting key 98

**Step 7:-** Consider seventh key  $k = 87$

$$h(87, 0) = (87 \bmod 9 + 0) \bmod 9$$

$$= (6 + 0) \bmod 9$$

$$= 6$$

Since address space A[6] is already occupied, the next address space is calculated as:

$$h(87, 1) = (87 \bmod 9 + 1) \bmod 9$$

$$= (6 + 1) \bmod 9$$

$$= 7$$

Since address space A[7] is vacant, insert key 87 at this place.

|   | 0   | 1  | 2  | 3   | 4  | 5  | 6  | 7  | 8  |
|---|-----|----|----|-----|----|----|----|----|----|
| A | --- | 46 | 65 | --- | 22 | 31 | 49 | 87 | 98 |

Hash table after inserting key 87

**Step 8:-** Consider eighth key  $k = 12$

$$h(12, 0) = (12 \bmod 9 + 0) \bmod 9$$

$$= (3 + 0) \bmod 9$$

$$= 3$$

Since address space A[3] is vacant, insert key 12 at this place.

|   | 0   | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |
|---|-----|----|----|----|----|----|----|----|----|
| A | --- | 46 | 65 | 12 | 22 | 31 | 49 | 87 | 98 |

Hash table after inserting key 12

**Step 9:-** Consider ninth key  $k = 97$

$$h(97, 0) = (97 \bmod 9 + 0) \bmod 9$$

$$= (7 + 0) \bmod 9$$

$$= 7$$

Since address space A[7] is already occupied, the next address space is calculated as:

$$h(97, 1) = (97 \bmod 9 + 1) \bmod 9$$

$$= (7 + 1) \bmod 9$$

$$= 8$$

Since address space A[8] is already occupied, the next address space is calculated as:

$$h(97, 2) = (97 \bmod 9 + 2) \bmod 9$$

$$= (7 + 2) \bmod 9$$

$$= 0$$

Since address space A[0] is vacant, insert key 97 at this place.

|   |    |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|----|
|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |
| A | 97 | 46 | 65 | 12 | 22 | 31 | 49 | 87 | 98 |

Hash table after inserting key 97

## ii) Quadratic probing:

Quadratic probing operates by taking the original hash value and adding successive values of an arbitrary quadratic polynomial to the starting value.

The hash function used for quadratic probing is

$$h(k, i) = [k \bmod m + C_1 i + C_2 i^2] \bmod m$$

for  $i = 0, 1, 2, \dots, m-1$ .

Where

- $m$  is the size of the hash table.
- $i$  is the probe number.
- $C_1$  and  $C_2 \neq 0$  are auxiliary constants.

e.g.

Insert the keys 65, 22, 31, 49 into a hash table of size  $m = 9$  using quadratic probing with  $C_1 = 1$  and  $C_2 = 3$

|   |     |     |     |     |     |     |     |     |     |
|---|-----|-----|-----|-----|-----|-----|-----|-----|-----|
|   | 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   |
| A | --- | --- | --- | --- | --- | --- | --- | --- | --- |

Initial state of hash table

**Step 1:-** Consider first key  $k = 65$

$$h(65,0) = (65 \bmod 9 + 0 + 0) \bmod 9$$

$$= (2) \bmod 9$$

$$= 2$$

Since address space  $A[2]$  is vacant, insert key 65 at this place.

|   |     |     |    |     |     |     |     |     |     |
|---|-----|-----|----|-----|-----|-----|-----|-----|-----|
|   | 0   | 1   | 2  | 3   | 4   | 5   | 6   | 7   | 8   |
| A | --- | --- | 65 | --- | --- | --- | --- | --- | --- |

Hash table after inserting key 65

**Step 2:-** Consider second key  $k = 22$

$$h(22,0) = (22 \bmod 9 + 0 + 0) \bmod 9$$

$$= (4) \bmod 9$$

$$= 4$$

Since address space  $A[4]$  is vacant, insert key 22 at this place.

|   |     |     |    |     |    |     |     |     |     |
|---|-----|-----|----|-----|----|-----|-----|-----|-----|
|   | 0   | 1   | 2  | 3   | 4  | 5   | 6   | 7   | 8   |
| A | --- | --- | 65 | --- | 22 | --- | --- | --- | --- |

Hash table after inserting key 22

**Step 3:-** Consider third key  $k = 31$

$$h(31,0) = (31 \bmod 9 + 0 + 0) \bmod 9$$

$$= (4) \bmod 9$$

$$= 4$$

Since address space  $A[4]$  is already occupied, the next address space is calculated as:

$$h(31,1) = (31 \bmod 9 + 1 + 3 * 1^2) \bmod 9$$

$$= (4 + 1 + 3) \bmod 9$$

$$= 8$$

Since address space  $A[8]$  is vacant, insert key 31 at this place.

|   |     |     |    |     |    |     |     |     |    |
|---|-----|-----|----|-----|----|-----|-----|-----|----|
|   | 0   | 1   | 2  | 3   | 4  | 5   | 6   | 7   | 8  |
| A | --- | --- | 65 | --- | 22 | --- | --- | --- | 31 |

Hash table after inserting key 31

**Step 4:** Consider fourth key  $k = 49$

$$\begin{aligned}
 h(49, 0) &= (49 \bmod 9 + 0 + 0) \bmod 9 \\
 &= (4 + 0 + 0) \bmod 9 \\
 &= 4
 \end{aligned}$$

Since address space  $A[4]$  is already occupied, the next address space is calculated as:

$$\begin{aligned}
 h(49, 1) &= (49 \bmod 9 + 1 + 3 * 1^2) \bmod 9 \\
 &= (4 + 1 + 3) \bmod 9 \\
 &= 8
 \end{aligned}$$

Since address space  $A[8]$  is already occupied, the next address space is calculated as:

$$\begin{aligned}
 h(49, 2) &= (49 \bmod 9 + 2 + 3 * 2^2) \bmod 9 \\
 &= (4 + 2 + 12) \bmod 9 \\
 &= 0
 \end{aligned}$$

Since address space  $A[0]$  is vacant, insert key 49 at this place,

|   |    |     |    |     |    |     |     |     |    |
|---|----|-----|----|-----|----|-----|-----|-----|----|
|   | 0  | 1   | 2  | 3   | 4  | 5   | 6   | 7   | 8  |
| A | 49 | --- | 65 | --- | 22 | --- | --- | --- | 31 |

Hash table after inserting key 49

### (iii) Double hashing:

Double hashing is technique in which a second hash function is applied to the key when a collision occurs.

Double hashing represents an improvement over linear or quadratic probing.

The hash function used for double hashing is

$$\begin{aligned}
 h(k, i) &= [k \bmod m + i (k \bmod m')] \bmod m \\
 &\text{for } i = 0, 1, 2, \dots, m-1.
 \end{aligned}$$

Where

- $m$  is the size of the hash table.
- $i$  is the probe number.

- $m'$  is taken slightly less than  $m$ . It may be  $m-1$  or  $m-2$ .

e.g.

Insert the keys 65, 22, 31, 58 into a hash table of size  $m=9$  and  $m'=7$  using double hashing.

|   |     |     |     |     |     |     |     |     |     |
|---|-----|-----|-----|-----|-----|-----|-----|-----|-----|
|   | 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   |
| A | --- | --- | --- | --- | --- | --- | --- | --- | --- |

Initial state of hash table

**Step 1:-** Consider first key  $k = 65$

$$\begin{aligned}
 h(65,0) &= (65 \bmod 9 + 0 \times 65 \bmod 7) \bmod 9 \\
 &= (2) \bmod 9 \\
 &= 2
 \end{aligned}$$

Since address space  $A[2]$  is vacant, insert key 65 at this place.

|   |     |     |    |     |     |     |     |     |     |
|---|-----|-----|----|-----|-----|-----|-----|-----|-----|
|   | 0   | 1   | 2  | 3   | 4   | 5   | 6   | 7   | 8   |
| A | --- | --- | 65 | --- | --- | --- | --- | --- | --- |

Hash table after inserting key 65

**Step 2:-** Consider second key  $k = 22$

$$\begin{aligned}
 h(22,0) &= (22 \bmod 9 + 0 \times 22 \bmod 7) \bmod 9 \\
 &= (4) \bmod 9 \\
 &= 4
 \end{aligned}$$

Since address space  $A[4]$  is vacant, insert key 22 at this place.

|   |     |     |    |     |    |     |     |     |     |
|---|-----|-----|----|-----|----|-----|-----|-----|-----|
|   | 0   | 1   | 2  | 3   | 4  | 5   | 6   | 7   | 8   |
| A | --- | --- | 65 | --- | 22 | --- | --- | --- | --- |

Hash table after inserting key 22

**Step 3:-** Consider third key  $k = 31$

$$\begin{aligned}
 h(31,0) &= (31 \bmod 9 + 0 \times 31 \bmod 7) \bmod 9 \\
 &= (4) \bmod 9 \\
 &= 4
 \end{aligned}$$

Since address space  $A[4]$  is already occupied, the next address space is calculated as:

$$\begin{aligned}
 h(31, 1) &= (31 \bmod 9 + 1 \times 31 \bmod 7) \bmod 9 \\
 &= (4 + 3) \bmod 9 \\
 &= 7
 \end{aligned}$$

Since address space A[7] is vacant, insert key 31 at this place.

|   | 0   | 1   | 2  | 3   | 4  | 5   | 6   | 7  | 8   |
|---|-----|-----|----|-----|----|-----|-----|----|-----|
| A | --- | --- | 65 | --- | 22 | --- | --- | 31 | --- |

Hash table after inserting key 31

**Step 4:-** Consider fourth key  $k = 58$

$$\begin{aligned}
 h(58, 0) &= (58 \bmod 9 + 0 \times 58 \bmod 7) \bmod 9 \\
 &= (4 + 0) \bmod 9 \\
 &= 4
 \end{aligned}$$

Since address space A[4] is already occupied, the next address space is calculated as:

$$\begin{aligned}
 h(58, 1) &= (58 \bmod 9 + 1 \times 58 \bmod 7) \bmod 9 \\
 &= (4 + 2) \bmod 9 \\
 &= 6
 \end{aligned}$$

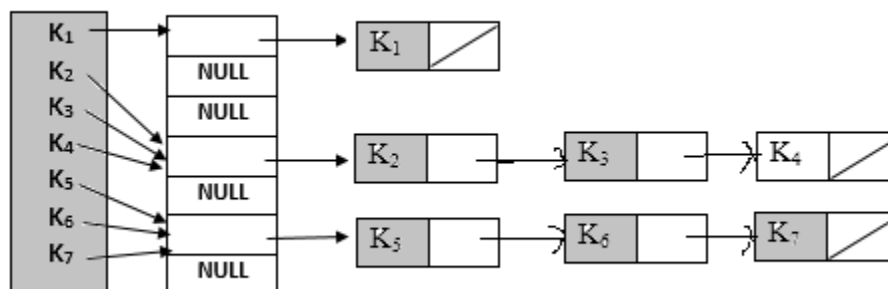
Since address space A[6] is vacant, insert key 49 at this place.

|   | 0   | 1   | 2  | 3   | 4  | 5   | 6  | 7  | 8   |
|---|-----|-----|----|-----|----|-----|----|----|-----|
| A | --- | --- | 65 | --- | 22 | --- | 58 | 31 | --- |

Hash table after inserting key 58

## 2. Chaining method

In chaining, all the elements whose keys hash to the address space are put in a linked list.



In chaining method insertion and deletion operations are easy to perform.